Supplementary Information (SI)

ALO: A Screen Reader Architecture for Bengali-English Code-Switching with Temporal Event Coalescing

MD. MAHIR LABIB[1], ANM JUBAER[1], AMIRUL ISLAM[1], PARTHA SUTRADHAR[1], BIJAN PAUL[1], AKM SHAHARIAR AZAD RABBY[1], FUAD RAHMAN[2]

[1] Apurba Technologies Ltd., Dhaka, Bangladesh (Emails: mdmahirlabib@gmail.com, jubaerad1@gmail.com, amirulislam00026@gmail.com, partharaj.dev@gmail.com, bijancse@gmail.com, rabby@apurbatech.com)

[2] Apurba Technologies, CA, USA (Email: fuad@apurbatech.com)

Corresponding authors: Md. Mahir Labib (mdmahirlabib@gmail.com) and Bijan Paul (bijancse@gmail.com)

1. Overview

This supplementary document provides additional resources to support the reproducibility and evaluation of the ALO screen reader Windows system. The materials include the application installer, an evaluation summary (aggregate findings already reported in the main manuscript), user manual, installation guidelines, and system documentation diagrams. Corresponding authors (consistent with the main article): Bijan Paul (bijancse@gmail.com) and Md. Mahir Labib (mdmahirlabib@gmail.com), Apurba Technologies Ltd., Dhaka, Bangladesh.

Supplementary materials accompanying this submission are available in a single public repository (installer, installation guidelines, user manual, evaluation summary, and diagrams): https://github.com/apurbatech/alo-paper-supplementary

2. Provided Materials

Application Installer : Executable Windows application for deployment and testing
Installation Guidelines : Step-by-step setup instructions
Evaluation Summary: Aggregate study findings and high-level benchmark results summarized in Section 5 (not per-participant raw logs or full multi-configuration dumps).
User Manual : Detailed usage instructions and hotkey reference
Diagrams: High-quality diagrams (with Mermaid source where available) for implementation traceability.

3. Development Milestones

The milestone table records the Windows desktop project plan under which ALO was developed. Only the Windows desktop system was implemented and evaluated in the accompanying paper.

No.	Milestone	Deliverables
1	**Inception Report**	SRS, Design Document, Methodology, Workshop Report, Workplan
2	**Milestone 1**	Windows v.1; Application Set 1: Chrome, Firefox, Edge, MS Word, MS Excel, PDF Reader; External TTS integration
3	**Milestone 2**	Windows v.2; Application Set 1 v.2 (continued support); Native and External TTS/API integration
4	**Milestone 3**	Windows v.2.1; Application Set 1 v.2.1; Application Set 2: Gmail, Facebook, WhatsApp, Messenger, Zoom, YouTube; Native/External TTS and STT/API integration
5	**Milestone 4**	Windows v.3; Full support for Application Sets 1 & 2; Native/External TTS and STT finalized; 1 Research paper published; Fine-tuning based on UAT; Public release & documentation

4. User Manual Summary

System requirements:

Minimum : Windows 10 (64-bit), 4 GB RAM, 1GB disk space

Recommended : Windows 11 (64-bit), 8 GB RAM, 1GB disk space
Dependencies : .NET 4.8 runtime, Windows UI Automation, Bengali TTS engine

Key features:

- Keyboard-driven navigation with application-specific hotkey sets
- Automatic Bengali-English and Romanized Bengali language detection
- Priority-stratified temporal event coalescing (50 ms window)
- Pre-indexed virtual buffer with O(log n) heading and element navigation
- Plugin architecture for Zoom, WhatsApp, Word, Excel, PowerPoint, Facebook, YouTube and Web/Gmail
- Guided progressive onboarding via context-sensitive audio cues
- Deterministic COM object lifecycle management

A full hotkey reference and plugin configuration guide is provided in the user manual.

5. User Study and Evaluation Summary

This section summarizes the controlled task-based study already reported in the main manuscript. It does not include per-participant raw logs, full interaction traces, or separate multi-configuration benchmark tables beyond the aggregate findings below. A controlled task-based study was conducted with 15 visually impaired participants recruited through accessibility communities in Dhaka, Bangladesh. Participants had an average of 4.2 years of screen reader experience and were fluent Bengali speakers. The study compared ALO against NVDA and JAWS across three task categories: document navigation, mixed-language text input and Bengali web browsing. ALO, NVDA and JAWS were not configured with functionally equivalent Bengali-English pipelines; results characterize the tested configurations rather than capability-matched bilingual setups.

Key findings:

- Participants provided an overall satisfaction score **8.5 out of 10**
- ALO's language-routing path averaged 105.0 ms (SD = 18.7 ms); NVDA's 134.6 ms value is engine-switch only. Different measurement scopes: descriptive only; no direct percentage advantage is claimed.
- Under the evaluated default configurations, NVDA and JAWS fell back to English synthesis on unmarked Bengali content (including chat messages and web portals), whereas ALO handled the same content automatically.
- Romanized Bengali (e.g., "ami", "kemon acho") was correctly routed by ALO's lexicon stage; neither evaluated comparator configuration provided automatic Romanized Bengali detection.
- All 15 of 15 participants preferred ALO for Bengali and mixed Bengali-English content (exact binomial 95% CI: 78.2-100%); 13 of 15 preferred ALO for general navigation and Bengali web browsing (86.7%, 95% CI: 59.5-98.3%).

- System benchmark feedback on Bengali TTS clarity and navigation responsiveness was consistently positive across the evaluated hardware configurations.

6. Supplementary Resources

All supplementary resources are provided via one repository. Corresponding authors: Bijan Paul (bijancse@gmail.com) and Md. Mahir Labib (mdmahirlabib@gmail.com).

Repository (all materials): https://github.com/apurbatech/alo-paper-supplementary

7. Reproducibility Statement

Components needed to install and inspect the Windows system, including the installer, configuration instructions and evaluation summary materials, are provided with this submission. The plugin architecture and documented service interfaces enable further extension and integration with other assistive technologies. Benchmark measurements were collected using Windows Performance Monitor counters and high-precision .NET timing APIs (Stopwatch.GetTimestamp, DateTime.UtcNow) across three hardware configurations as described in the paper. Full raw multi-configuration dumps and per-participant logs are not packaged in this summary document.

8. Additional Notes

Demonstration materials referenced for transparency are linked with the submission package when available. The system is designed to be extensible and adaptable for future research in bilingual screen reading, multilingual accessibility and Windows UI Automation-based assistive technology. Only the Windows desktop system is evaluated in the accompanying paper.

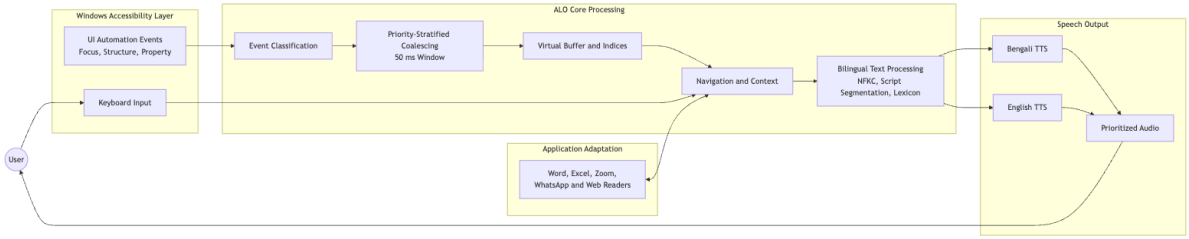
9. Detailed Implementation and UML Diagrams

The main manuscript now uses six consolidated scientific figures to foreground the research architecture, mechanisms, and evaluation results. The diagrams below preserve the more detailed UML and implementation views removed during that consolidation. They are provided for design traceability, reproducibility, and maintainability, and should be interpreted as supplementary engineering documentation rather than additional independent experimental evidence.

The former windows_focus_change_flow diagram is not repeated because it is pixel-identical to the UI Automation focus-change diagram reproduced as Supplementary Figure S6.

9.1 Legacy End-to-End System Architecture

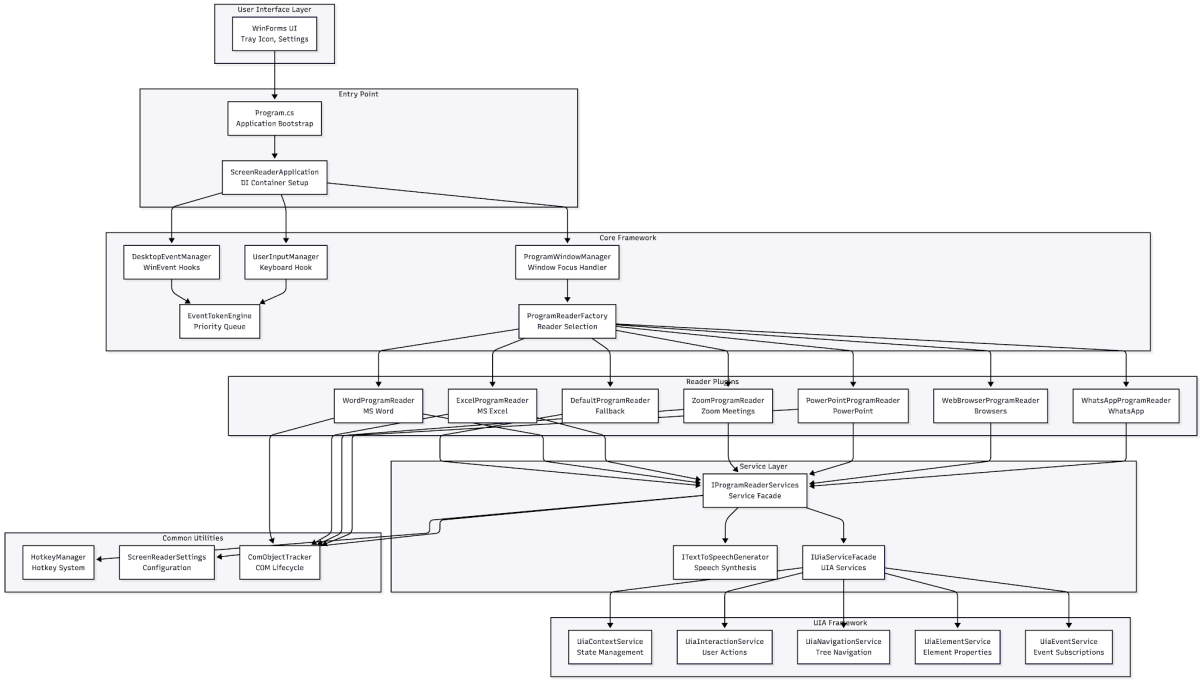
This view preserves the original deployment and data-path decomposition. Windows UI Automation events and user input enter the event-management layer, pass through generic or application-specific readers, and converge on bilingual speech output. It is a structural implementation map; the consolidated architecture and quantitative evidence remain in the main manuscript.



Supplementary Figure S1. Legacy end-to-end system architecture retained for implementation traceability.

9.2 Complete Component and Dependency Architecture

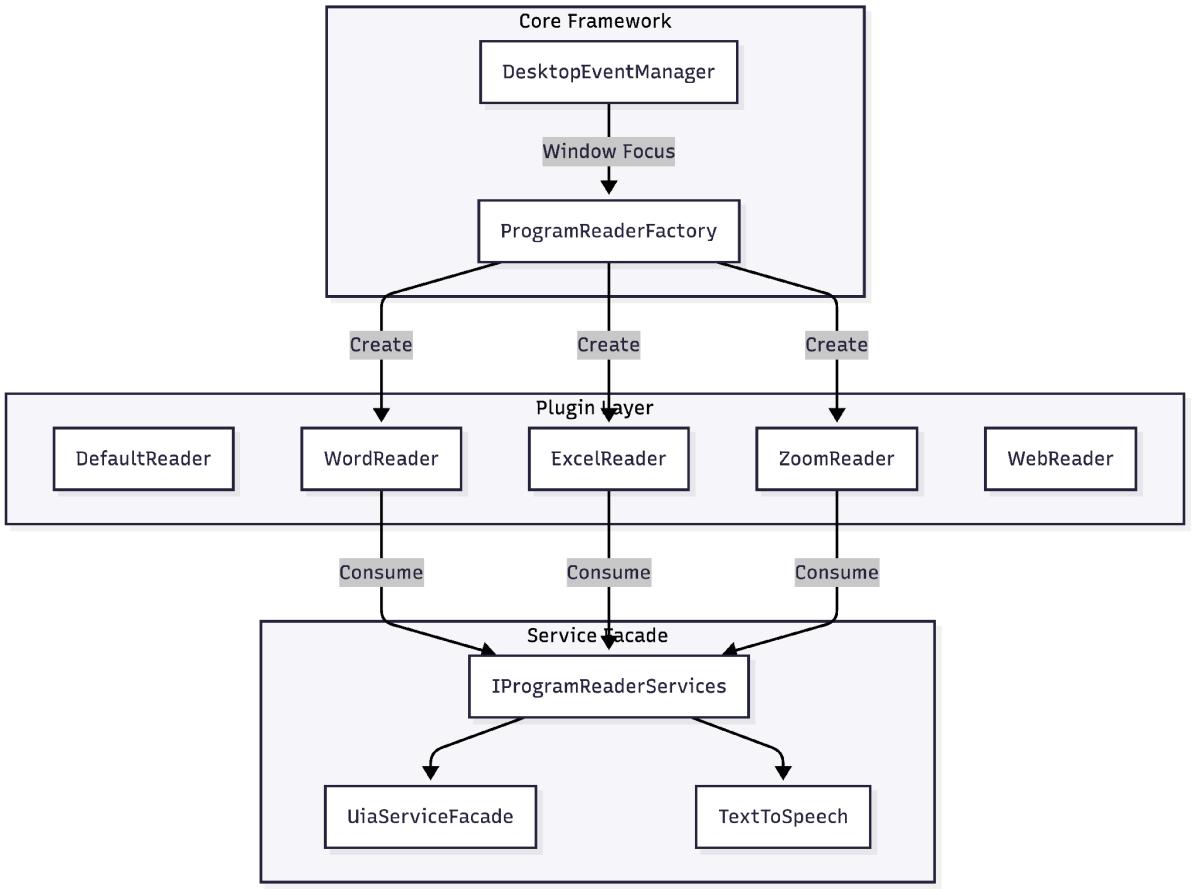
The expanded architecture shows the boundaries among the application shell, dependency-injection container, event and hotkey services, Windows UI Automation framework, shared reader services, utilities, and application plugins. Dependencies are intentionally directed toward shared abstractions so that readers can be substituted without coupling one plugin to another.



Supplementary Figure S2. Complete component and dependency architecture of the ALO implementation.

9.3 Plugin Boundary and Shared-Service Contract

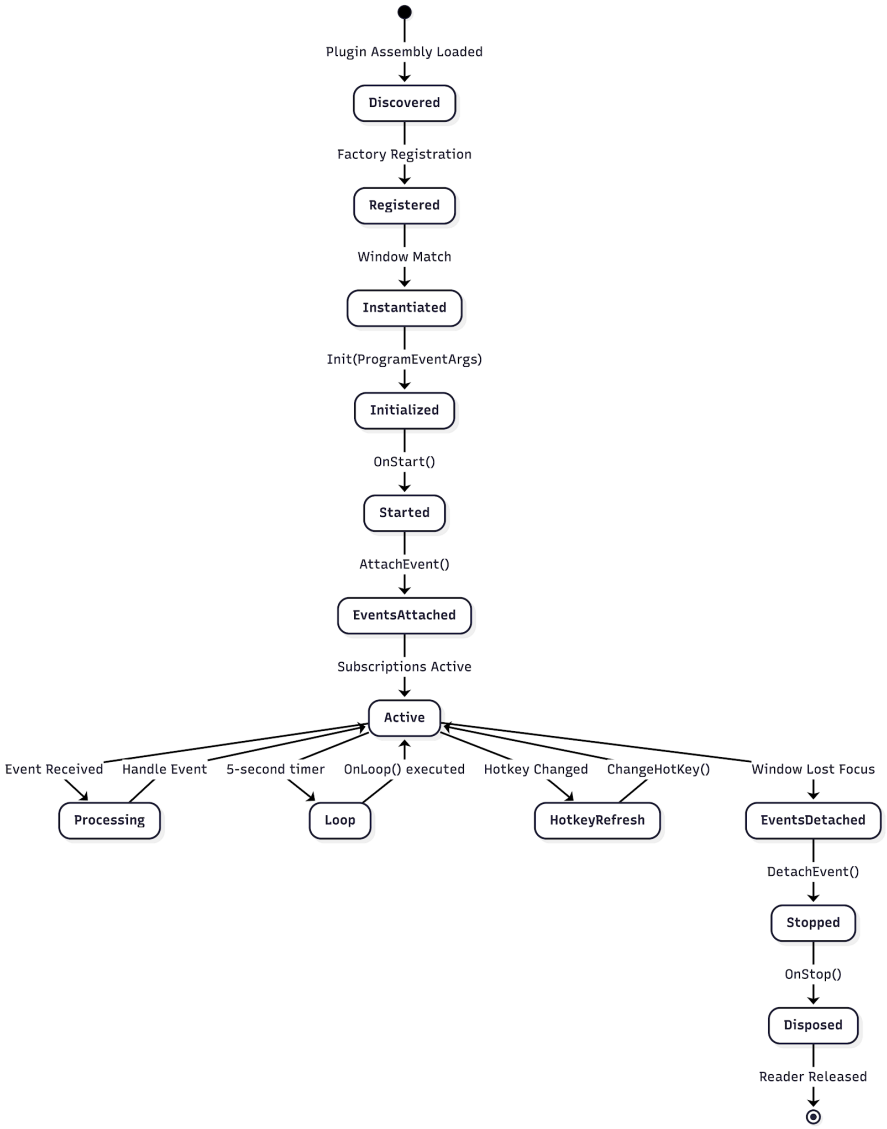
The plugin manager selects the reader associated with the active application and supplies common services through stable interfaces. Application readers therefore specialize interaction logic while reusing event, speech, UIA, configuration, and logging facilities. Direct plugin-to-plugin dependencies are avoided, limiting the effect of application-specific changes.



Supplementary Figure S3. Plugin boundary, selection logic, and shared-service contract.

9.4 Plugin Lifecycle State Machine

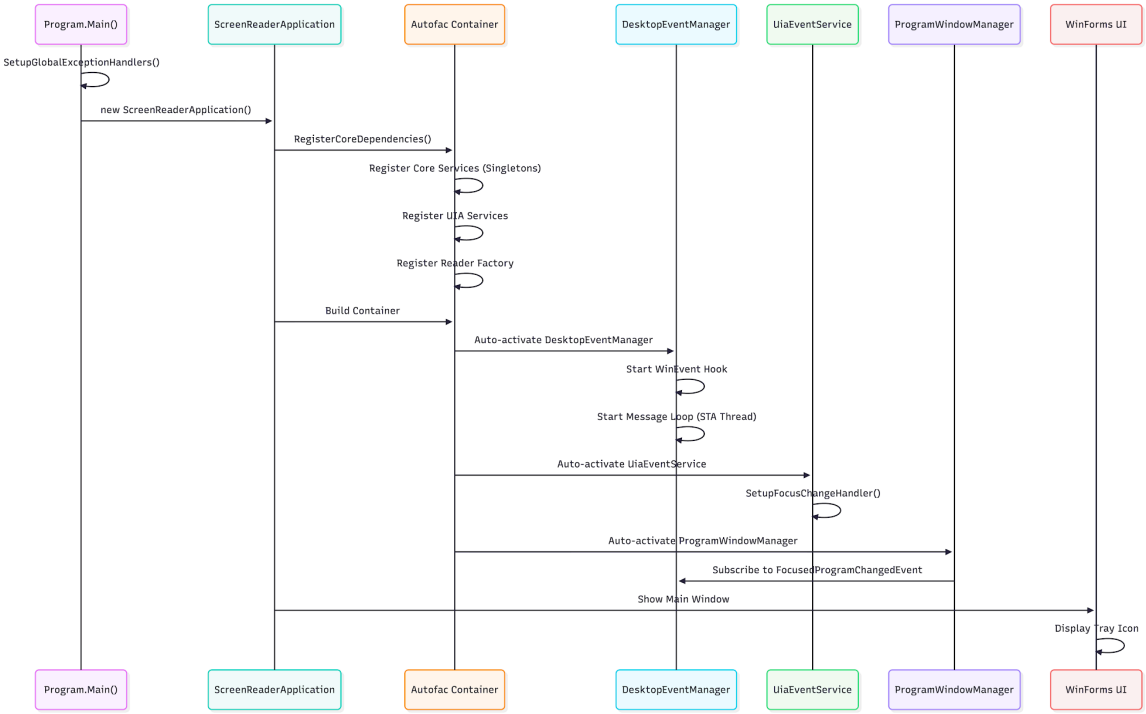
This lifecycle state machine details discovery, registration, instantiation, initialization, active event processing, detachment, stopping, and disposal. The essential invariant is that every active subscription, timer, hook, or native resource receives a corresponding cleanup action before the reader leaves the active state.



Supplementary Figure S4. Plugin discovery, activation, event handling, deactivation, and disposal lifecycle.

9.5 Application Startup Sequence

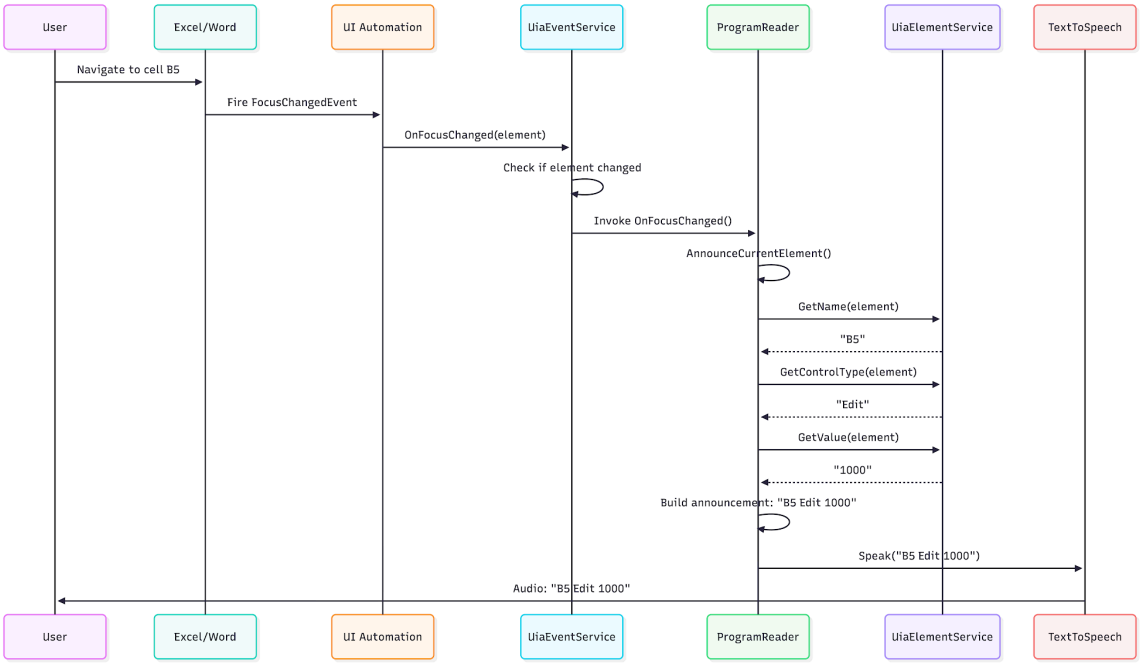
The startup sequence records the order in which configuration, dependency registration, event management, UI Automation services, window tracking, plugins, and the application shell become available. Event processing begins only after its required services are initialized, preventing partially constructed readers from receiving focus or keyboard events.



Supplementary Figure S5. Ordered startup sequence for dependency registration and service activation.

9.6 UI Automation Focus-Change Sequence

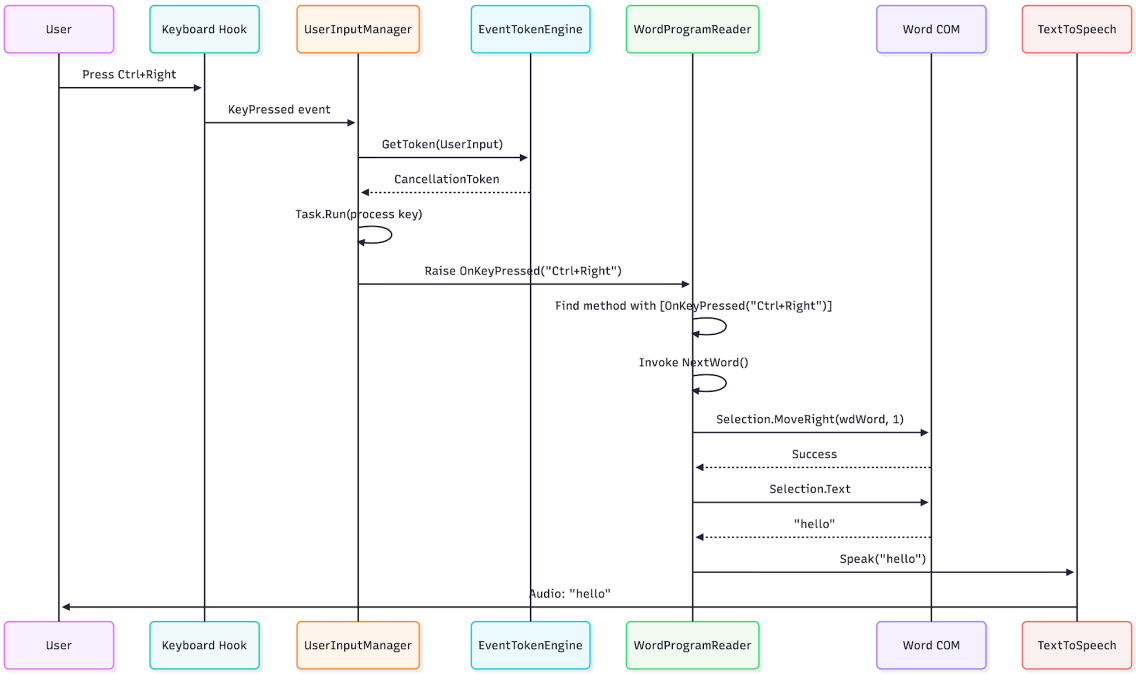
A Windows focus-change notification is received by the UIA event service, associated with the active application reader, enriched with cached control properties, and converted into an announcement request. The speech service then applies suppression and language-routing rules before synthesis. This sequence exposes the call-level responsibilities hidden by the main paper's higher-level pipeline.



Supplementary Figure S6. Detailed UI Automation focus-change and announcement sequence.

9.7 Hotkey Processing and Cancellation Sequence

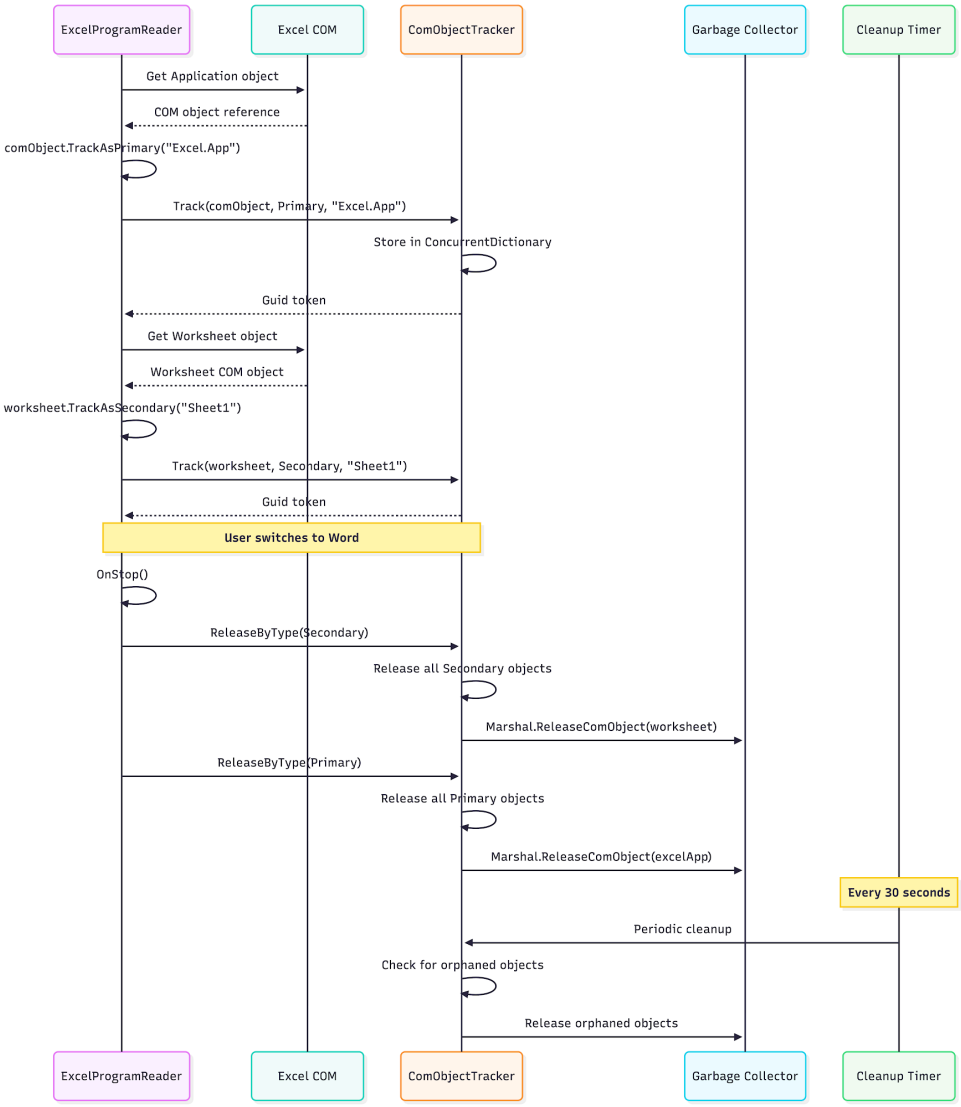
The keyboard hook converts a recognized key combination into a semantic command rather than invoking application logic directly. The command is dispatched to the active reader with a cancellation token, after which UIA or native automation work produces the requested announcement. Cancellation prevents stale results from being spoken when a newer navigation action supersedes them.



Supplementary Figure S7. Hotkey capture, semantic-command dispatch, cancellation, and speech sequence.

9.8 COM Ownership and Release Sequence

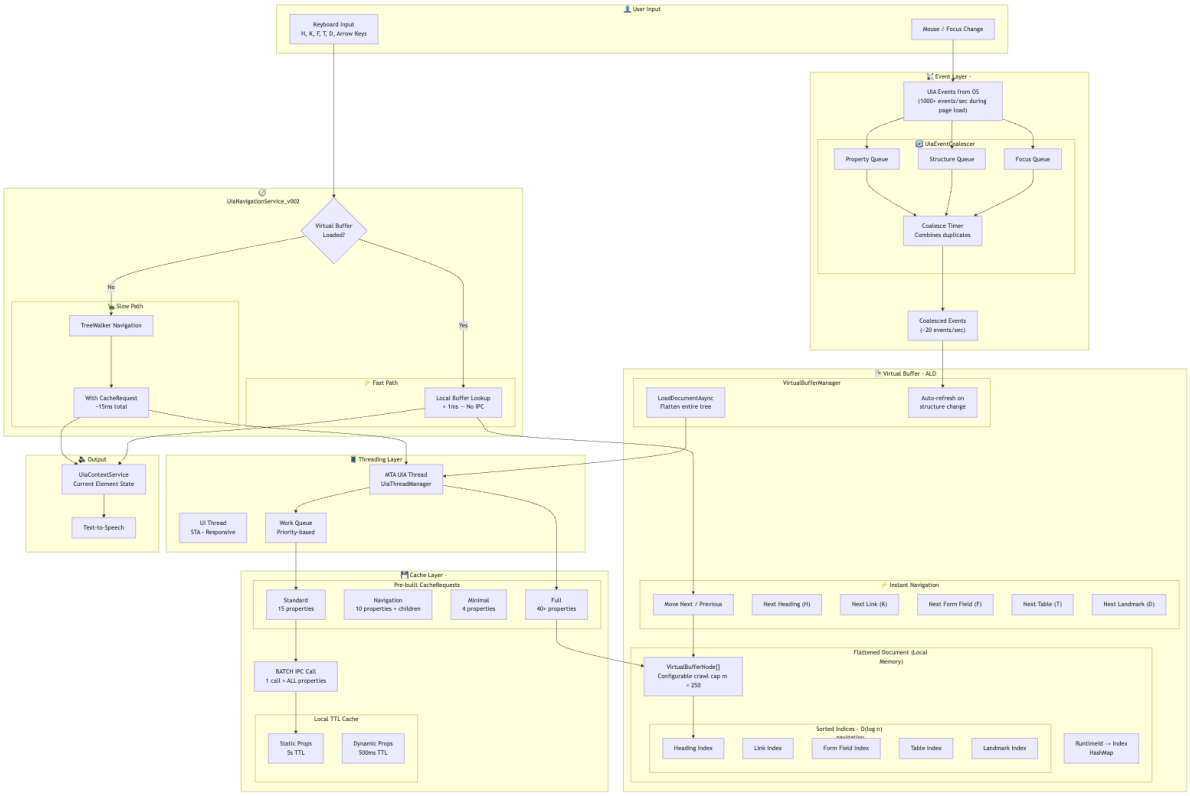
The sequence distinguishes primary automation objects from secondary objects created during traversal. Each acquired resource is registered with the tracker and released at the end of its ownership scope; a final sweep handles resources retained by exceptional paths. This design is a leak-mitigation mechanism and does not imply that every external accessibility provider is itself leak-free.



Supplementary Figure S8. Native COM-resource ownership, tracking, and deterministic release sequence.

9.9 Detailed UIA Cache and Navigation Flow

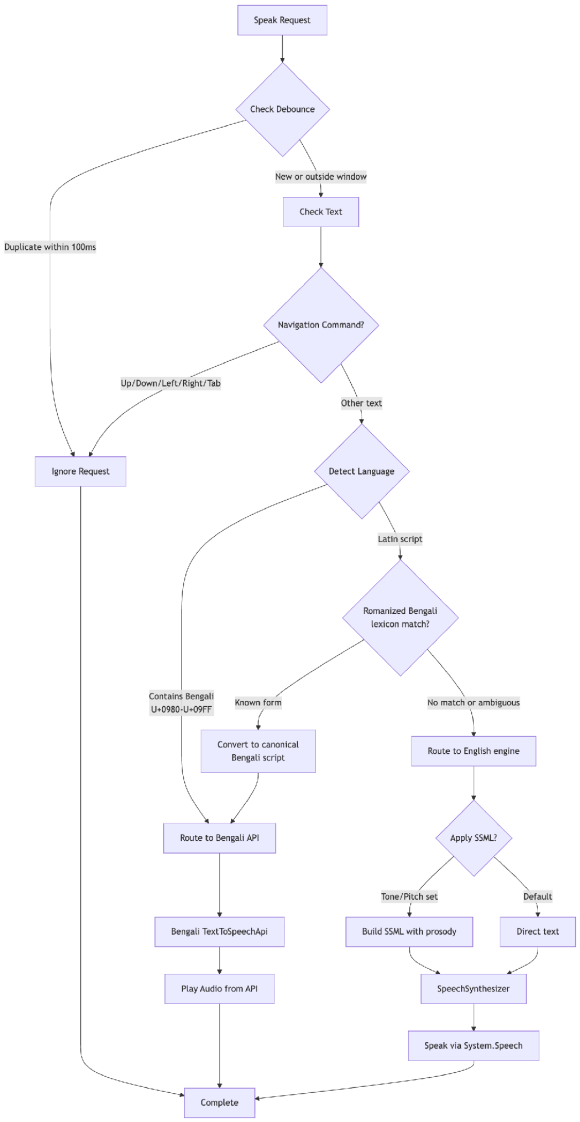
Incoming focus and structure events are coalesced and serialized before a CacheRequest obtains the properties needed for navigation. A bounded flattened buffer (m = 250) supports typed indices and O(log n) lookup, while a direct UIA path remains available when the cached representation is incomplete. This figure supplies the implementation detail behind the navigation optimization described in the main manuscript.



Supplementary Figure S9. Detailed UIA event coalescing, cache construction, typed indexing, and navigation flow.

9.10 Legacy Detailed Bilingual TTS Routing

A speech request first passes through duplicate and navigation suppression. Text containing Bengali Unicode characters is routed to the Bengali service; recognized Romanized Bengali is normalized before following the same route. Other text is sent to the local English synthesizer, with SSML prosody applied when requested. This operational flow is retained as implementation detail; latency values in the main paper follow the explicitly stated end-to-end and engine-switch measurement scopes.



Supplementary Figure S10. Detailed bilingual text-to-speech routing and speech-suppression decision flow.